

---

# Do Coding Agents Keep Their Word? Design-Commitment Drift and Context Management in Multi-Turn Repository Work

---

Shriya Sudhakar  
Cornell University  
ss3576@cornell.edu

## Abstract

Software engineering with coding agents is increasingly conversational: requirements arrive over multiple turns, agents state design decisions as they work, and later requirements may force those decisions to evolve. Existing coding benchmarks primarily evaluate the final artifact, making it difficult to distinguish an agent that remains coherent with its earlier design from one that silently abandons it. We introduce *design-intent fidelity*, an evaluation framework that treats an agent’s own explicit design commitments as trajectory-specific constraints and checks those commitments against the code produced over subsequent turns. Because violating an earlier commitment can reflect either healthy adaptation or genuine loss of intent, we separate cross-turn violations into requirement changes, announced agent revisions, and silent drift. We apply the framework to eight multi-file SWE-bench Verified tasks under six context-management conditions, producing 144 coding trajectories and 1,227 extracted commitments. Raw commitment violations substantially overstate silent drift: many violations reflect later requirements or explicit design revisions, while unacknowledged architectural drift is comparatively rare. We also introduce Design Commitment Carryover (DCC), a context-management strategy that exposes the agent to a persistent ledger of its own prior commitments. DCC yields fewer observed cross-turn violations than the sharded baseline, whereas strategies that restate the user’s requirements do not show the same reduction. This total effect can arise in two ways: the ledger may help the agent preserve commitments it already made, or it may change which commitments the agent makes in the first place. DCC does not reduce commitment volume and does not make commitments detectably more hedged; its commitments are only directionally narrower. These checks argue against a simple fewer-or-safer-commitments explanation, although they do not fully isolate the retention pathway. Stronger fidelity to the agent’s own stated design also does not translate into higher task resolution or closer adherence to maintainer derived patch constraints.

## 1 Introduction

Coding agents increasingly work through evolving conversations rather than a single complete specification. A user and an agent inspect a repository, discuss implementation choices, add requirements, and revise code over several turns. Final-state coding benchmarks do not directly measure whether the resulting implementation remains coherent with design decisions the agent stated earlier in that trajectory.

Prior work shows substantial degradation when tasks are distributed across turns [3, 7], but outcome metrics do not reveal how an agent’s design changes over time. Two trajectories may reach the same pass/fail outcome even if one preserves its earlier architecture and the other silently abandons it.

SWE-Shield similarly shows that task success and design-constraint satisfaction can diverge in a single-diff setting [9].

We ask: **as context accumulates, do coding agents remain faithful to their own stated design commitments, and can context-management strategies change that behavior?** A violation alone is not enough to answer this question. Later requirements may correctly supersede an earlier decision, or the agent may explicitly revise its design. We therefore separate these cases from unacknowledged drift.

Our contributions are threefold:

1. **A design-intent-fidelity instrument and violation taxonomy.** We extract self-stated design commitments, track them across subsequent repository states, distinguish Neutral/Satisfied/Violated states, and separate same-turn delivery gaps from requirement changes, announced revisions, and silent drift.
2. **An evaluation across six context-management strategies.** Across eight multi-file SWE-bench Verified tasks, we compare standard context conditions with DCC, which exposes the agent to a de-duplicated ledger of its own prior commitments. We measure the total effect of this intervention on later violations without assuming that it operates only through memory.
3. **Evidence that coherence and correctness come apart.** DCC lowers self-stated violations, but its advantage does not translate into higher task resolution or closer adherence to maintainer-derived gold-patch constraints.

## 2 Related Work

Laban et al. [3] introduced sharded simulations for studying multi-turn degradation and found especially large losses on generative, non-decomposable tasks; MT-Sec reports related degradation in secure coding [7]. Observation-masking work compares context-management strategies on resolution, cost, and latency [4], while SlopCodeBench and architecture-quality judging measure properties of produced code over longer horizons [6, 8]. We use related multi-turn manipulations but measure a different dependent variable: whether the implementation remains consistent with design commitments generated earlier in the same trajectory.

SWE-Shield mines design constraints from pull-request history and verifies patches against those external constraints [9]. We reuse the general verifier mechanism but change the constraint source and temporal role: commitments come from the agent’s own words and are tracked across multiple code states. Text-level belief-drift metrics and per-turn editing benchmarks are adjacent [5, 2], but do not connect an agent’s self-stated software-design commitments to its evolving repository states.

## 3 Measuring Design-Intent Fidelity

### 3.1 From agent statements to design commitments

We define **design-intent fidelity** as the extent to which an agent’s implementation remains consistent with design commitments that the agent itself stated earlier in the conversation.

The word *itself* is essential. We are not asking whether the agent chose the best architecture, or whether it chose the architecture a maintainer would prefer. We ask a narrower question: having said that it would build or preserve the code in a particular way, does it subsequently do so? The standard is generated by the agent, so it is trajectory-specific: it may differ across repeated runs of the same task and does not exist until the agent states it.

A **design commitment** is an explicit statement that constrains how some part of the implementation should be built, changed, or left unchanged. Following the constraint schema of Yu et al. [9], we represent each extracted commitment as

$$c = (p, d, \gamma, r, q),$$

where  $p$  is the problem being addressed,  $d$  is the design decision,  $\gamma$  is any condition under which it applies,  $r$  is the stated rationale, and  $q$  is a verbatim quote anchoring the commitment to the agent’s

own message. The quote prevents the evaluator from attributing an unstated design intention to the agent.

For example, in one printer task the agent states: *“Extend the shared \_coeff\_isneg helper . . . rather than adding separate per-printer special-casing.”* This commits the implementation to more than fixing sign detection. It specifies a structural choice: the behavior should be implemented through a shared abstraction rather than duplicated across printer-specific code.

After each assistant turn, a frozen extraction prompt identifies newly stated commitments and records them in this schema. We sample the extractor three times. Reconciliation votes only on whether the turn contains any commitment: if at most one of the three samples returns commitments, the turn yields none. Otherwise, among the samples that found commitments, we sort by the number extracted and keep the median-length sample wholesale. We do not semantically merge commitments across samples. Statements about activity rather than design, such as “I will run the tests next,” are filtered out because they do not constrain how the code should be implemented.

### 3.2 Following a commitment through code states

A commitment is not judged only at the end of a trajectory. Let  $S_t$  denote the cumulative repository state after turn  $t$ , and let commitment  $c_i$  be stated at turn  $t_i$ . We evaluate  $c_i$  against every subsequent state

$$S_{t_i}, S_{t_i+1}, \dots, S_T,$$

so each commitment has a trajectory rather than a single verdict. For example,

$$\text{Neutral} \rightarrow \text{Satisfied} \rightarrow \text{Satisfied} \rightarrow \text{Violated}$$

means that the repository initially provided no evidence about the commitment, later implemented it, preserved it for another turn, and then became inconsistent with it.

Each commitment–state pair is evaluated with a two-step verifier that sees the extracted commitment, the cumulative diff relative to the untouched repository, and the list of changed files. First, the verifier asks whether the current code state is informative about the commitment. If the relevant area has not yet been touched, the verdict is **Neutral**. For negative commitments such as “leave X untouched,” applicability instead asks whether the current diff could have violated the protected area. Second, if the commitment is applicable, the verifier asks whether the code honors it (**Satisfied**) or contradicts it (**Violated**). The verifier may still return Neutral if the diff does not settle the question.

Formally,

$$J(c_i, S_t) \in \{\text{Neutral}, \text{Satisfied}, \text{Violated}\}.$$

Neutral is not partial failure; it means the current code state is not informative enough to evaluate the commitment. We therefore exclude Neutral judgments from violation-rate denominators.

The verifier returns *applicable*, a verdict in  $\{\text{Satisfied}, \text{Neutral}, \text{Violated}\}$ , and one to three sentences of evidence citing the specific diff hunks that ground the verdict. Final states use three samples with majority vote; logged ties resolve to Neutral. Intermediate states use one sample. Across the 1,227 commitments, this yields 2,477 state judgments: 1,227 final-state judgments and 1,250 intermediate-state judgments.

### 3.3 Same-turn and cross-turn violations

A violated commitment can arise in two temporally different ways, which we separate before computing drift.

A **same-turn violation** occurs when a commitment is stated and already contradicted by the code produced in that same turn. For example, an agent announces an Oracle-specific override of `_alter_column_type_sql`, but the resulting diff never touches the Oracle schema file. There was never an earlier code state in which the commitment held. We therefore interpret this pattern as a *delivery gap*: the agent announced more implementation scope than it delivered within the turn.

A **cross-turn violation** occurs when a commitment is Satisfied at one code state and becomes Violated at a later state:

$$\text{Satisfied at } t \rightarrow \dots \rightarrow \text{Violated at } t + k.$$

This transition indicates that the implementation once honored the stated decision and later stopped doing so. Cross-turn violations are the events that require causal interpretation for drift.

The `FULL` and `CONCAT` conditions provide all requirements within a single assistant turn, so they can exhibit same-turn delivery gaps but cannot exhibit cross-turn violations by construction.

### 3.4 Not every cross-turn violation is drift

A commitment becoming violated does not by itself imply agent failure. When requirements arrive over several turns, the correct implementation may need to depart from an earlier design. An agent that mechanically preserves every early commitment would often be less useful, not more. We therefore separate the observable transition from its cause and classify every cross-turn violation into one of three categories.

**Requirement change.** A later user requirement invalidates or supersedes the earlier commitment. In one trajectory, the agent writes that it “Left `str/pretty` printers untouched, since the bug report is specifically about LaTeX.” The next shard reveals that the user wants all three printers fixed. Extending the fix violates the earlier commitment literally, but the change is externally induced and appropriate.

**Agent-initiated revision.** No new requirement directly forces the change, but the agent revises its own design and explicitly acknowledges that revision. Because the change is surfaced in the conversation, the user can see that the design has moved and can object if needed.

**Silent drift.** The implementation becomes inconsistent with a prior commitment without a superseding requirement and without an explicit acknowledgment of the design change. In the shared-helper example above, the agent commits to extending `_coeff_isneg`, later implements sign detection separately in `str.py`, and never mentions the architectural departure. A user reading only the conversation would still believe the shared abstraction was being preserved.

Only the third category corresponds to the unacknowledged design drift targeted by this paper. The first two are forms of adaptation. This distinction is fundamental to the construct: a raw cross-turn violation rate conflates beneficial revision with loss of stated intent.

We classify each cross-turn violation with up to three independent classifier calls. Of the 69 violations, 55 receive three parseable votes, 8 receive two, 4 receive one, and 2 receive none. The latter two remain unresolved; all other cases are assigned by the available votes. Thus the 69 events comprise 34 requirement changes, 19 announced revisions, 14 silent-drift cases, and 2 unresolved cases.

### 3.5 What the metric does and does not measure

Design-intent fidelity measures self-consistency, not design quality. A perfectly faithful agent can consistently implement a poor design and score well. Conversely, an agent may violate an earlier commitment because it discovers a better approach. Our taxonomy separates explicit adaptation from silent drift, but it does not turn self-consistency into a correctness metric.

We therefore evaluate correctness independently using the official SWE-bench harness on the final code state. We also compare self-stated commitments with design constraints mined from each task’s gold patch and verified using the same constraint-checking mechanism. These external constraints are not treated as uniquely correct designs; rather, they provide a second reference point for asking whether fidelity to the agent’s own plan tracks fidelity to a maintainer-derived design. As we show later, the two measurements behave differently.

## 4 Experimental Design

### 4.1 Tasks and requirement sequencing

We select eight SWE-bench Verified tasks [1] using a non-decomposability criterion adapted from Laban et al. [3]. Each selected task must create revision pressure: later information should be capable

of forcing the agent to revisit code or a design decision it already made, rather than merely requesting an independent addition.

Before running the main grid, we place tasks into three evidence groups and analyze those groups separately where relevant:

- **Statement-based (4 tasks):** every shard is drawn from real problem-statement text, re-sequenced so that a generalizing requirement arrives later.
- **Patch-based (3 tasks):** the final shard contains information absent from both the problem statement and issue thread and is therefore derived from the gold patch. Because this is a weaker source of “user” intent, we keep these tasks separate in analysis.
- **Consistency-driven (1 task):** no requirement is withheld; revision pressure instead comes from maintaining consistency with a derived file.

Appendix A.1 lists all eight task instances, repositories, evidence groups, shard counts, and forced-revision shards.

## 4.2 Context-management conditions

We re-implement the three-role simulation used by Laban et al. [3]. The coding agent is `claude-sonnet-5`; it can inspect the repository using `read` and `grep` operations and can edit files. The harness records diffs and commits at the end of each turn. A user simulator, `claude-haiku-4-5`, sees only information that has been revealed up to the current turn.

We evaluate six context-management conditions: five from Laban et al. [3] and our DCC intervention (Table 1).

Table 1: Context-management conditions. Prior user and assistant prose persists across turns in all multi-turn conditions; tool calls and tool results do not. Repository state persists in the workspace.

| Condition | Input construction                                                                                                            |
|-----------|-------------------------------------------------------------------------------------------------------------------------------|
| FULL      | One turn with the original SWE-bench <code>problem_statement</code> verbatim.                                                 |
| CONCAT    | One turn with all shards concatenated verbatim as a bulleted list.                                                            |
| SHARDED   | One turn per shard; shard 1 is verbatim and later shards are paraphrased by the leak-proof user simulator.                    |
| RECAP     | SHARDED plus a final recap turn containing all shards verbatim as bullets and asking the agent to ensure the fix covers them. |
| SNOWBALL  | Each new-shard paraphrase is followed by a verbatim recap of all previously revealed shards.                                  |
| DCC       | SHARDED plus, from turn 2 onward, a de-duplicated [DESIGN LEDGER] containing the agent’s prior commitments.                   |

The key contrast is between strategies that restate the *user’s requirements* and Design-Commitment Carryover (DCC), which also exposes the agent to its *own prior commitments*. In every multi-turn condition, prior user messages and the assistant’s final prose persist as plain text, while tool calls and tool results do not. Repository state persists in the workspace, and the assistant is instructed to end each turn by summarizing its changes and design decisions.

From turn 2 onward, DCC appends a [DESIGN LEDGER] containing the agent’s previously extracted commitments. A newly extracted commitment is omitted when its Jaccard word overlap with an existing ledger entry exceeds 0.6. Appendix A.2 gives the exact ledger template.

**Formation versus retention.** A lower violation rate under DCC can arise through two mechanisms. The ledger may help the agent preserve commitments it has already made, or it may change the commitments the agent forms on later turns. We therefore do not interpret a lower violation rate as evidence of improved memory alone. In addition to fidelity, we compare commitment volume, breadth, and hedging under DCC and SHARDED to test whether obvious changes in commitment formation explain the effect. Section 6 reports this analysis.

### 4.3 End-to-end protocol

For each task-condition pair, we run three independent trajectories, yielding

$$8 \text{ tasks} \times 6 \text{ conditions} \times 3 \text{ repeats} = 144 \text{ conversations.}$$

Across these conversations, the extractor produces 1,227 design commitments.

Each trajectory follows the same protocol. The harness reveals the requirement information specified by the current condition, after which the coding agent inspects the repository and edits the code using read, grep, and edit tools. The agent cannot execute code or run tests. At the end of each turn, the harness records the repository state and the agent’s final natural-language response, then extracts any newly stated design commitments.

Trajectory length is fixed by the task rather than chosen adaptively. Each task receives exactly one turn per shard, giving two to four turns depending on the task. The RECAP condition adds one final recap turn after all shards have been revealed.

After the conversation ends, each extracted commitment is evaluated against every applicable repository state from the turn in which it was stated onward. This produces a sequence of Neutral, Satisfied, or Violated judgments for each commitment. We separate commitments that are already violated in the turn in which they are stated from commitments that transition from Satisfied to Violated on a later turn. Cross-turn violations are then classified as requirement changes, announced agent revisions, silent drift, or unresolved when no classifier response is parseable. Final task correctness is evaluated separately using the official SWE-bench Docker harness.

### 4.4 Three-family final-state evaluation

Every final-state commitment is sent to three judge families: `claude-sonnet-5`, `gemini-3.5-flash`, and `gpt-5.5`. Claude is the primary judge and uses a three-sample majority vote. Gemini and GPT each provide one independent cross-family label. Gemini returns 1,207 valid verdicts out of 1,227 commitments and GPT returns 1,225; 1,205 commitments have valid final-state verdicts from all three families. The external families evaluate final states only, not intermediate states. Main tables use the primary Claude label, while the external families are used for robustness checks.

### 4.5 Primary estimand and task-level comparisons

Our primary unit of aggregation is the conversation rather than the individual commitment, since commitments within a trajectory are correlated. For conversation  $r$ , we define

$$\text{VR}_r = \frac{V_r}{S_r + V_r},$$

where  $V_r$  and  $S_r$  are applicable Violated and Satisfied commitments; Neutral judgments are excluded. For direct DCC versus SHARDED comparisons, we average repeats within each task and report

$$\Delta_j = \overline{\text{VR}}_{j,\text{DCC}} - \overline{\text{VR}}_{j,\text{Sharded}}.$$

This keeps the inferential unit aligned with the eight tasks supporting cross-task generalization.

## 5 Instrument Validation

On a 24-turn pilot set, extraction reaches 87.5% raw agreement with human annotation ( $\kappa = 0.684$ ), with commitment-level recall of 23/24 and no fabricated commitments. A fresh-data audit finds 0/58 fabricated commitments across 12 additional turns. Blind cross-family re-extraction reaches  $\kappa = 0.895$ .

At the final state, the primary Claude judge agrees with Gemini on 91.0% of valid overlapping items ( $\kappa = 0.738$ ) and with GPT on 84.3% ( $\kappa = 0.619$ ); all three families are unanimous on 82.3% of items with three valid verdicts. The two external judges disagree with Claude in opposite directions, providing no evidence of systematic same-family leniency. Detailed disagreement counts are reported in Appendix A.

Table 2: Commitment violations at the final code state. Cross-turn violations are classified by cause; only *silent drift* is unacknowledged. Two of 69 cross-turn violations remain unresolved because no classifier response was parseable, one under SHARDED and one under RECAP. FULL and CONCAT have no cross-turn rate because all requirements are delivered within one assistant turn.

| Condition | Violation rate      |                       | Cross-turn cause (count) |                |                     |
|-----------|---------------------|-----------------------|--------------------------|----------------|---------------------|
|           | same-turn           | cross-turn            | req. change              | agent revision | <b>silent drift</b> |
| FULL      | 6.0% (3/50)         | N/A                   | N/A                      | N/A            | N/A                 |
| CONCAT    | <b>17.0%</b> (8/47) | N/A                   | N/A                      | N/A            | N/A                 |
| SHARDED   | 4.5% (3/66)         | <b>14.3%</b> (18/126) | 11                       | 3              | <b>3</b>            |
| RECAP     | 5.2% (4/77)         | 12.0% (23/192)        | 10                       | 6              | <b>6</b>            |
| SNOWBALL  | 7.5% (5/67)         | 12.7% (19/150)        | 9                        | 6              | <b>4</b>            |
| DCC       | 4.9% (3/61)         | <b>6.4%</b> (9/140)   | 4                        | 4              | <b>1</b>            |

To test discriminant validity, we mine 4–7 constraints per task from the gold patch and evaluate them against the same final diffs, yielding 864 judgments (517 non-Neutral). Maintainer-derived constraints are violated more often than self-stated commitments (approximately 30–47% versus 6–14%), and the two are uncorrelated at the task level (Spearman  $\rho = 0.24$ ,  $p = .57$ ). Gold patches represent one acceptable design rather than unique ground truth, so we interpret these violations as divergence from the maintainer-derived design rather than necessarily as software defects.

## 6 Results

### 6.1 Raw violations overstate silent drift

The first result is that commitment violations are not rare, but genuinely silent drift is much rarer than the raw rate suggests. Under the three multi-turn baseline conditions, cross-turn violation rates range from 12.0% to 14.3% (Table 2). DCC is lower at 6.4%. Read naively, these numbers would suggest that agents frequently abandon earlier design commitments.

The taxonomy in Section 3.4 changes that interpretation. Among 69 cross-turn violations, 34 are attributed to later requirement changes, 19 to explicit agent-initiated revisions, 14 to silent drift, and 2 remain unresolved because no classifier vote was parseable. Requirement-change violations are expected under our forced-revision design: the later requirement makes the earlier narrow commitment obsolete, and the agent correctly overrides itself. Announced revisions likewise expose the change to the user. Only silent drift leaves the conversational record asserting a design that the later code no longer follows.

Silent drift is also qualitatively architectural. The recurring pattern is not simply a forgotten line-level edit; it is an agent that commits to a shared abstraction and later re-implements equivalent logic separately in multiple modules. The shared-helper example in Appendix C is the modal case in the most violation-heavy task.

The effect is uneven across tasks. Within the statement-based group, the reported cross-turn rate reaches 19.2% [10.6, 28.2], but excluding the most violation-heavy task lowers the group to 11.4%. We therefore interpret the data as evidence of concentration in particular task structures rather than a smooth gradient across evidence groups.

Silent drift occurs in 14 of 608 applicable cross-turn commitments (2.30%; Wilson 95% CI [1.38, 3.83]). At the conversation level, 10 of 85 eligible multi-turn conversations (11.8%) contain at least one silent-drift event.

### 6.2 What context is carried forward matters

DCC produces fewer observed cross-turn violations than the sharded baseline. The aggregate rate falls from 14.3% under SHARDED to 6.4% under DCC. By comparison, RECAP and SNOWBALL, which restate user-side information, remain at 12.0% and 12.7%. This establishes an effect of ledger exposure on the measured violation rate. It does not yet tell us whether the ledger mainly helps the agent keep existing commitments or changes which commitments it makes.

The violation decomposition argues against a pure memory explanation. Relative to SHARDED, requirement-change violations fall from 11 to 4, while silent drift falls from 3 to 1 and announced revisions move from 3 to 4. If the ledger only reminded the agent of commitments it would otherwise forget, we would expect the clearest change in silent drift. Instead, much of the reduction occurs among commitments that would later become incompatible with new requirements.

The data argue against the simplest formation-only explanation. DCC produces 250 commitments, compared with 227 under SHARDED, so the lower violation rate is not caused by making fewer commitments. DCC commitments are directionally less broad (22.4% vs. 29.5%; Fisher OR= 0.69,  $p = .093$ ) but not more hedged (8.4% vs. 5.7%;  $p = .29$ ). Within the commitments that enter the analysis denominator, these differences nearly disappear: 19.3% vs. 22.2% are broad and 7.9% vs. 7.1% are hedged. Thus DCC’s advantage is not explained by fewer or obviously safer commitments.

This does not prove a pure retention mechanism. The ledger could still change unmeasured properties of commitment formation, and the drop in requirement-change violations (11 to 4) suggests that formation may play some role. The current experiment therefore supports a mixed interpretation: DCC reduces violations even after the obvious commitment-volume, scope, and hedging explanations are checked, but it does not cleanly identify how much of the effect comes from retention versus subtler changes in commitment formation.

### 6.3 Greater self-consistency does not imply greater correctness

The strongest qualification to the DCC result is that fidelity to the agent’s own design does not purchase task success. Resolution is low in every condition: SNOWBALL resolves 3/24 runs; FULL and CONCAT, 2/24 each; RECAP and SHARDED, 1/24 each; and DCC, 0/24. All nine resolutions occur in the statement-based task group, while the patch-based and consistency-driven tasks resolve no runs under any condition.

The same grid therefore produces a clear spread in design-intent fidelity but almost no corresponding spread in correctness. In particular, DCC has the lowest aggregate cross-turn violation rate and zero resolved runs. This is not evidence that DCC harms correctness. The experiment is too small and task resolution is near a floor. It does, however, show that improving self-consistency is not sufficient to improve SWE-bench success.

The gold-constraint analysis reinforces the distinction. DCC is lowest on self-stated violations but descriptively highest on violations of maintainer-derived gold-patch constraints (45.4%), with no reported significant difference from SHARDED. In other words, the commitment ledger makes the agent more likely to follow what it previously said without making that stated design more likely to match the maintainer-derived design.

This is not a contradiction in the metric; it is the distinction the metric is designed to expose. A system can become more internally coherent while remaining wrong, or while consistently pursuing a design that differs from the maintainer’s. Context mechanisms that preserve intent therefore need to be paired with mechanisms that improve or re-evaluate the quality of that intent.

## 7 Conclusion

Design-intent fidelity tracks whether coding agents follow commitments they stated earlier in the same trajectory. Raw violations overstate drift because many reflect later requirements or explicit self-revision. DCC reduces observed cross-turn violations relative to the sharded baseline. This advantage is not explained by making fewer commitments or by a detectable increase in hedging, and scope differences are small within the analysis denominator. The intervention may still act through both retention and subtler changes in commitment formation. DCC also does not improve task resolution or agreement with maintainer-derived constraints. Preserving a plan, choosing a plan, and executing a correct plan are related but distinct problems.

## References

- [1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.



- [2] Manav Nitin Kapadnis, Lawanya Baghel, Atharva Naik, and Carolyn Rosé. ChartEditBench: Evaluating grounded multi-turn chart editing in multimodal language models. *arXiv preprint arXiv:2602.15758*, 2026.
- [3] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*, 2025.
- [4] Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. The complexity trap: Simple observation masking is as efficient as LLM summarization for agent context management. In *Deep Learning for Code (DL4Code) Workshop at NeurIPS*, 2025.
- [5] Praveen Kumar Myakala, Manan Agrawal, and Rahul Manche. BeliefShift: Benchmarking temporal belief consistency and opinion drift in LLM agents. *arXiv preprint arXiv:2603.23848*, 2026.
- [6] Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks. *arXiv preprint arXiv:2603.24755*, 2026.
- [7] Ruchit Rawal, Jeffrey Yang Fan Chiang, Chihao Shen, Jeffery Siyuan Tian, Aastha Mahajan, Tom Goldstein, and Yizheng Chen. Benchmarking correctness and security in multi-turn code generation. *arXiv preprint arXiv:2510.13859*, 2025.
- [8] Kirill Vasilevski, Ximing Dong, Benjamin Rombaut, Milad Soltany, Ruochen Deng, Jiahuei Lin, Arthur Leung, Dayi Lin, Boyuan Chen, Shaowei Wang, and Ahmed E. Hassan. Beyond correctness: Enhancing architectural reasoning in code LLMs via scalable labeling with agentic judgment. *arXiv preprint arXiv:2606.14948*, 2026.
- [9] Kai Yu, Zhenhao Zhou, Junhao Zeng, Ying Wang, Xueying Du, Zhiqiang Yuan, Junwei Liu, Ziyu Zhou, Yujia Wang, Chong Wang, and Xin Peng. Does pass rate tell the whole story? Evaluating design constraint compliance in LLM-based issue resolution. *arXiv preprint arXiv:2604.05955*, 2026.

## A Extended Method and Results

### A.1 Main task set

Table 3: Repository tasks used in the main grid. “FR shard” denotes the shard that introduces the forced-revision requirement.

| Instance      | Repository        | Group       | Shards | FR shard | Marker   |
|---------------|-------------------|-------------|--------|----------|----------|
| django-11138  | django/django     | statement   | 4      | 4        | strong   |
| django-11400  | django/django     | statement   | 3      | 3        | strong   |
| sphinx-9461   | sphinx-doc/sphinx | statement   | 3      | 3        | weak     |
| sympy-14248   | sympy/sympy       | statement   | 3      | 3        | strong   |
| django-15629  | django/django     | patch       | 3      | 3        | strong   |
| astropy-13398 | astropy/astropy   | patch       | 3      | 3        | strong   |
| pylint-4551   | pylint-dev/pylint | patch       | 3      | 3        | moderate |
| sympy-16597   | sympy/sympy       | consistency | 2      | N/A      | N/A      |

### A.2 DCC ledger construction

From turn 2 onward, DCC appends the following block to the user message:

```
[DESIGN LEDGER - reminder of the design commitments YOU have stated
so far in this conversation:
1. (turn N) <decision> (applies: <condition>)
...
Honor these commitments in further changes, or state explicitly that
you are revising one and why.]
```

A newly extracted decision is omitted from the ledger when its Jaccard word overlap with any existing ledger entry exceeds 0.6.

### A.3 Supporting metrics

We additionally track five supporting metrics: self-revision churn (lines added by an earlier turn and removed by a later turn, using diff provenance), duplication among added lines (repeated 5-line shingles), cyclomatic complexity and maintainability by code state, correctness using the official SWE-bench Docker harness on the final diff, and anticipation using per-task markers for the withheld requirement that were fixed before the main grid was run. The assistant model exposes no temperature control, so repeated trajectories capture sampling variation under the available API configuration. Every model call is logged.

### A.4 Task concentration and correlated violation echoes

The statement-based task with the strongest non-decomposable structure, a parallel printer reimplement, is the most violation-heavy task in every condition, with reported rates between 20% and 40%. Excluding it, the remaining statement-based tasks have an 8.1% violation rate, close to the 8.9% rate in the other evidence groups. We therefore avoid claiming a general gradient from task category to violation rate.

Violations can also appear as correlated echoes of the same underlying change. Grouping by the state turn at which a violation first flips, the manuscript reports that 69 commitment-level cross-turn violations collapse to 53 distinct events, with an echo multiplier of 1.2–1.6 $\times$  and a maximum of 3 $\times$  within one conversation. This dependence is another reason the main analysis should not treat commitment labels as independent samples.

The 69-to-53 collapse is a separate grouping operation from cause classification: it groups commitment-level violations by the turn on which they first flip, so repeated commitment violations caused by the same code change count as one event. The two unresolved cause labels remain part of the 69 commitment-level violations.

### A.5 Same-turn overcommitment

CONCAT has an elevated same-turn violation rate relative to FULL (17.0% versus 6.0%; 8 versus 3 events). Excluding the concentrated printer task, the reported rates are 11.1% and 7.9%, respectively. SNOWBALL also shows an elevated same-turn gap under the previous estimators. Individual inspection indicates that these events are unimplemented announced scope rather than design reversals. A full requirement list presented at once appears to elicit more ambitious design statements than the agent completes within one turn.

Relatedly, 28/144 conversations end without any code edits, including 10/24 CONCAT runs and a concentration in one proposal-style task.

### A.6 Anticipation

In 35–48% of multi-turn runs by condition, the agent’s diff touches the withheld requirement’s pre-registered marker before the reveal turn. Anticipated runs have a lower reported violation rate (9.5% versus 13.1%). Because a repository can expose architectural regularities that imply unrevealed requirements, this moderator has no direct analogue in a single-function prompt where the conversation contains nearly all task-relevant information.

### A.7 Three-family judge robustness

Every final-state commitment is evaluated under three judge families: `claude-sonnet-5`, `gemma-3.5-flash`, and `gpt-5.5`. Claude is the primary judge and uses a three-sample majority vote; Gemini uses one sample with thinking disabled; GPT uses one sample with low reasoning effort. Agreement with the primary judge is 91.0% raw ( $\kappa = 0.738$ ) for Gemini and 84.3% ( $\kappa = 0.619$ ) for GPT, with all three families unanimous on 82.3% of items.

Directional disagreements move in opposite directions: GPT is stricter than Claude in the observed disagreements (95 vs. 11), whereas Gemini is more lenient (6 vs. 31). Gemini agrees with Claude more than Gemini and GPT agree with each other ( $\kappa = 0.738$  vs.  $0.645$ ). These patterns argue against a simple same-family leniency explanation for the primary judge.

Gemini yields 1,207 valid final-state verdicts, GPT yields 1,225, and 1,205 commitments have valid verdicts from all three families. The external judges do not evaluate intermediate states.

### A.8 Gold-constraint reanalysis

Across the same 144 trajectories, gold-patch-derived constraints produce 864 final-state judgments, 517 of which are non-Neutral. DCC’s advantage on self-stated commitments does not transfer to these external constraints: it is descriptively highest on gold-constraint violations at 45.4%, with no reported statistically significant difference from SHARDED. RECAP remains descriptively close to the baseline on both axes.

Because a gold patch is one valid implementation rather than a unique design specification, we interpret this analysis as a contrast between self-consistency and maintainer-derived design alignment, not as a direct correctness test.

### A.9 Single-function comparison arm

We run Laban et al. [3]’s single-function sharded tasks through the same pipeline: eight HumanEval tasks and three LCB tasks, each divided into four or five shards, under FULL and SHARDED with three repeats (66 trajectories total). Cross-turn violations occur in 9.4% of commitments (30/318): 11.9% on HumanEval and 4.0% on LCB, compared with 14.3% on the repository tasks under SHARDED. Excluding the most violation-heavy repository task reduces the repository rate to 10.8%.

The reported Fisher comparison gives  $OR = 1.60$ ,  $p = .17$ , and the odds ratio falls to approximately 1.2 after excluding the concentrated repository task. We therefore regard the repository-versus-single-function difference as unresolved. FULL produces no same-turn delivery gaps in the single-function arm (0/89 versus 6.0% on repository tasks), while correctness differs little between full and sharded runs (33/33 versus 32/33 after normalization). These observations further separate design-intent drift, delivery gaps, and final task success.

## B Secondary Analyses

### B.1 Requirement dumps create delivery gaps rather than reversals

CONCAT has the highest same-turn violation rate, 17.0%, compared with 6.0% under FULL. Reading these violations individually, the current draft reports that they are announced scope that the agent does not complete within the turn, rather than designs that are implemented and then reversed. For example, an agent may announce fixes across three modules but edit only one before the turn ends.

This suggests a failure mode distinct from drift: presenting all requirements at once can elicit a broad implementation plan whose announced scope exceeds what the agent delivers in one turn. The behavior is therefore better described as overcommitment or a delivery gap than as loss of earlier intent.

Across the full grid, 28/144 conversations end with no code edits. These are concentrated in the single-turn conditions: 10 under CONCAT and 9 under FULL, versus 3 under SHARDED and 2 each under RECAP, DCC, and SNOWBALL. By task, `astropy-13398` accounts for 12 of the 28 no-edit runs, followed by `sphinx-9461` with 6 and `pylint-4551` with 4.

### B.2 Repositories can reveal withheld requirements

In 35–48% of multi-turn runs, depending on condition, the agent’s diff touches a pre-registered marker for the withheld requirement before that requirement is explicitly revealed. For example, after observing a shared operations pattern, the agent may fix all three database backends before the user mentions the additional backends. Runs that anticipate the withheld requirement show a lower reported violation rate (9.5% versus 13.1%).

This behavior highlights a difference between repository-scale and single-function sharding. The conversation is not the agent’s only source of information: the repository itself is a persistent and truthful context channel. Code structure can therefore partially reveal a later requirement before the user states it, weakening a clean separation between “known” and “withheld” information.

### B.3 Drift also appears in single-function tasks

As a comparison arm, we run the same pipeline on Laban et al. [3]’s sharded single-function tasks: eight HumanEval problems and three LCB problems, with four to five shards each, under FULL and SHARDED with three repeats. This produces 66 conversations.

Cross-turn violations also occur in this regime: 9.4% of commitments (30/318), compared with 14.3% on the repository tasks under the corresponding sharded setting. The reported odds ratio is 1.60 with Fisher’s exact  $p = .17$ ; excluding the repository task with the highest violation rate reduces the repository rate to 10.8% and the difference becomes small. We therefore treat the regime comparison as unresolved rather than claiming that repository work inherently causes more drift.

Correctness barely degrades under sharding in this comparison arm (32/33 versus 33/33 after normalization for task count), unlike the larger single-function degradation reported by Laban et al. [3]. This again suggests that success loss and design-intent drift need not move together.

## C Qualitative Examples

**Silent drift.** Turn 1: “Thread connection through to `get_tzname()` ...so it can make this decision.” By turn 3, the agent implements the fix entirely inside backend operations classes, leaving `get_tzname()` untouched. The final architecture differs from the stated commitment and the change is never acknowledged (primary verifier: 3/3 Violated).

**Silent drift: duplication over abstraction.** Turn 1: “Extend the shared `_coeff_isneg` helper ...rather than adding separate per-printer special-casing.” Turn 2 re-implements sign detection separately in `str.py`. The current analysis reports five instances of this pattern across one task’s runs in three conditions.

**Requirement change.** Turn 1: “Left `str/pretty` printers untouched, since the bug report is specifically about LaTeX.” The next shard reveals that the user wants all three printers fixed. The verifier flags a literal commitment violation, but the taxonomy labels the change as requirement-driven adaptation rather than drift.

**Same-turn delivery gap.** A single-turn CONCAT design section commits to “an Oracle-specific override of `_alter_column_type_sql` ...,” but the turn’s diff never touches the Oracle schema file.

**Commitment upgrade.** Turn 1 patches `even -> finite` while noting that “integer -> finite would be the more complete and consistent fix.” Turn 2 replaces the earlier patch accordingly. This is explicit self-revision that follows the agent’s own stated rationale rather than silently abandoning it.

## D Reproducibility Statement

We use the official SWE-bench Docker images for final task evaluation. The current recorded API costs are approximately \$245 for the main conversation grid, \$60 for classification and verification, \$12 for cross-family judge evaluation, and \$30 for gold-constraint and commitment-specificity reanalyses.

We will release the code and artifacts.

## E Figures

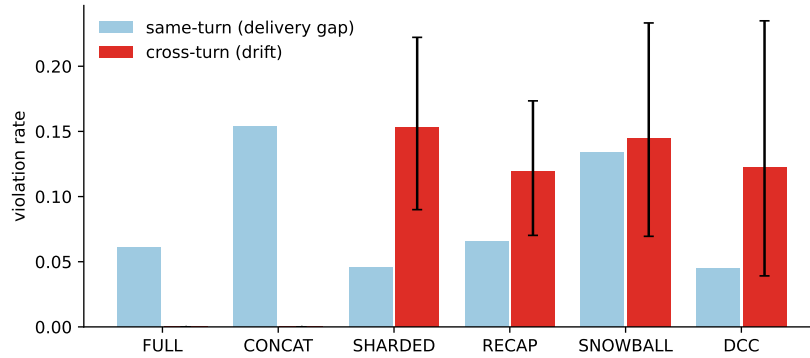


Figure 1: Same-turn delivery gaps and cross-turn violation rates by condition. Bars are per-conversation means; error bars on cross-turn rates are bootstrap 95% confidence intervals. Cross-turn violations include requirement changes and announced revisions as well as silent drift, so the cross-turn bar should not be interpreted as silent drift alone.

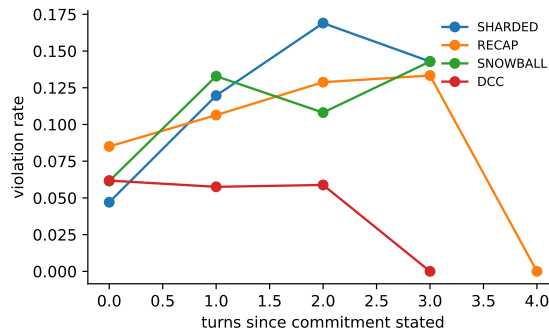


Figure 2: Supplementary commitment-level violation rate by turns elapsed since a commitment was stated. Each point pools applicable commitments within the elapsed-turn bin; this is a descriptive estimator and differs from the conversation-level primary analysis. Lines end when a condition has no commitments remaining in that bin, so tail values should be interpreted cautiously.